

In [12]: !pip install ucimlrepo

```
Requirement already satisfied: ucimlrepo in /usr/local/lib/python3.12/dist-packages (0.0.7)
Requirement already satisfied: pandas>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2.2.2)
Requirement already satisfied: certifi>=2020.12.5 in /usr/local/lib/python3.12/dist-packages (from ucimlrepo) (2026.2.25)
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.0.2)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.0.0->ucimlrepo) (2026.1)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas>=1.0.0->ucimlrepo) (1.17.0)
```

```
In [13]: import pandas as pd
import numpy as np
from ucimlrepo import fetch_ucirepo

from sklearn.model_selection import train_test_split
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import MinMaxScaler

#read in datasets; example that follows is for the Pima Indians Diabetes Dataset; f
#code provided by UCI ML to read in the datas
diabetes_df = pd.read_csv("diabetes.csv")
X_diab = diabetes_df.drop("Outcome", axis=1)
y_diab = diabetes_df["Outcome"]

# fetch dataset
glass_identification = fetch_ucirepo(id=42)

# data (as pandas dataframes)
X_glass = glass_identification.data.features
y_glass = glass_identification.data.targets

# metadata
print(glass_identification.metadata)

# variable information
print(glass_identification.variables)

print("Original Data:")
print(diabetes_df.head())

print("Original Data:")
print(pd.concat([X_glass, y_glass], axis=1).head())

# Display the classes and the no of samples in each type
print("Glass class distribution:")
```

```

print(y_glass.iloc[:, 0].value_counts().sort_index())

print("Diabetes class distribution:")
print(y_diab.value_counts().sort_index())

# Check missing values
print("Glass missing values:")
print(X_glass.isna().sum())

print("Diabetes missing values:")
print(X_diab.isna().sum())

# Replace impossible zeros with NaN (Diabetes only)
zero_as_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
X_diab_fixed = X_diab.copy()
for col in zero_as_missing:
    if col in X_diab_fixed.columns:
        X_diab_fixed[col] = X_diab_fixed[col].replace(0, np.nan)

# set up the train and test splits
Xg_train, Xg_test, yg_train, yg_test = train_test_split(
    X_glass, y_glass.iloc[:, 0],
    test_size=0.30, random_state=42, stratify=y_glass.iloc[:, 0]
)

Xd_train, Xd_test, yd_train, yd_test = train_test_split(
    X_diab_fixed, y_diab,
    test_size=0.30, random_state=42, stratify=y_diab
)

# Impute missing values (median)
imp_g = SimpleImputer(strategy="median")
Xg_train_imp = pd.DataFrame(imp_g.fit_transform(Xg_train), columns=Xg_train.columns)
Xg_test_imp = pd.DataFrame(imp_g.transform(Xg_test), columns=Xg_test.columns)

imp_d = SimpleImputer(strategy="median")
Xd_train_imp = pd.DataFrame(imp_d.fit_transform(Xd_train), columns=Xd_train.columns)
Xd_test_imp = pd.DataFrame(imp_d.transform(Xd_test), columns=Xd_test.columns)

# Normalizing all feature values to lie in the range 0 to 1.
scaler_g = MinMaxScaler()
Xg_train_norm = pd.DataFrame(scaler_g.fit_transform(Xg_train_imp), columns=Xg_train_imp.columns)
Xg_test_norm = pd.DataFrame(scaler_g.transform(Xg_test_imp), columns=Xg_test_imp.columns)

scaler_d = MinMaxScaler()
Xd_train_norm = pd.DataFrame(scaler_d.fit_transform(Xd_train_imp), columns=Xd_train_imp.columns)
Xd_test_norm = pd.DataFrame(scaler_d.transform(Xd_test_imp), columns=Xd_test_imp.columns)

print("Glass normalized sample:")
print(Xg_train_norm.head())

print("Diabetes normalized sample:")
print(Xd_train_norm.head())

scaler_vis_d = MinMaxScaler()
X_diab_normalized_df = pd.DataFrame(

```

```
scaler_vis_d.fit_transform(X_diab_fixed.fillna(X_diab_fixed.median(numeric_only
columns=X_diab_fixed.columns
)

scaler_vis_g = MinMaxScaler()
X_glass_normalized_df = pd.DataFrame(
    scaler_vis_g.fit_transform(X_glass.fillna(X_glass.median(numeric_only=True))),
    columns=X_glass.columns
)
```

```
{'uci_id': 42, 'name': 'Glass Identification', 'repository_url': 'https://archive.ics.uci.edu/dataset/42/glass+identification', 'data_url': 'https://archive.ics.uci.edu/static/public/42/data.csv', 'abstract': 'From USA Forensic Science Service; 6 types of glass; defined in terms of their oxide content (i.e. Na, Fe, K, etc)', 'area': 'Physics and Chemistry', 'tasks': ['Classification'], 'characteristics': ['Multivariate'], 'num_instances': 214, 'num_features': 9, 'feature_types': ['Real'], 'demographics': [], 'target_col': ['Type_of_glass'], 'index_col': ['Id_number'], 'has_missing_values': 'no', 'missing_values_symbol': None, 'year_of_dataset_creation': 1987, 'last_updated': 'Thu Aug 10 2023', 'dataset_doi': '10.24432/C5WW2P', 'creators': ['B. German'], 'intro_paper': None, 'additional_info': {'summary': 'Vina conducted a comparison test of her rule-based system, BEAGLE, the nearest-neighbor algorithm, and discriminant analysis. BEAGLE is a product available through VRS Consulting, Inc.; 4676 Admiralty Way, Suite 206; Marina Del Ray, CA 90292 (213) 827-7890 and FAX: -3189. In determining whether the glass was a type of "float" glass or not, the following results were obtained (# incorrect answers):\r\n\r\nType of Sample -- Beagle -- NN -- DA\r\n\r\nWindows that were float processed (87) -- 10 -- 12 -- 21\r\n\r\nWindows that were not: (76) -- 19 -- 16 -- 22\r\n\r\n\r\nThe study of classification of types of glass was motivated by criminological investigation. At the scene of the crime, the glass left can be used as evidence...if it is correctly identified!', 'purpose': None, 'funded_by': None, 'instances_represent': None, 'recommended_data_splits': None, 'sensitive_data': None, 'preprocessing_description': None, 'variable_info': '1. Id number: 1 to 214\r\n2. RI: refractive index\r\n3. Na: Sodium (unit measurement: weight percent in corresponding oxide, as are attributes 4-10)\r\n4. Mg: Magnesium\r\n5. Al: Aluminum\r\n6. Si: Silicon\r\n7. K: Potassium\r\n8. Ca: Calcium\r\n9. Ba: Barium\r\n10. Fe: Iron\r\n11. Type of glass: (class attribute)\r\n-- 1 building_windows_float_processed\r\n-- 2 building_windows_non_float_processed\r\n-- 3 vehicle_windows_float_processed\r\n-- 4 vehicle_windows_non_float_processed (none in this database)\r\n-- 5 containers\r\n-- 6 tableware\r\n-- 7 headlamps', 'citation': None}}
```

	name	role	type	demographic	description \
0	Id_number	ID	Integer	None	None
1	RI	Feature	Continuous	None	refractive index
2	Na	Feature	Continuous	None	Sodium
3	Mg	Feature	Continuous	None	Magnesium
4	Al	Feature	Continuous	None	Aluminum
5	Si	Feature	Continuous	None	Silicon
6	K	Feature	Continuous	None	Potassium
7	Ca	Feature	Continuous	None	Calcium
8	Ba	Feature	Continuous	None	Barium
9	Fe	Feature	Continuous	None	Iron
10	Type_of_glass	Target	Categorical	None	None

	units	missing_values
0	None	no
1	None	no
2	weight percent in corresponding oxide	no
3	weight percent in corresponding oxide	no
4	weight percent in corresponding oxide	no
5	weight percent in corresponding oxide	no
6	weight percent in corresponding oxide	no
7	weight percent in corresponding oxide	no
8	weight percent in corresponding oxide	no
9	weight percent in corresponding oxide	no
10	None	no

Original Data:

Pregnancies Glucose BloodPressure SkinThickness Insulin BMI \

0	6	148	72	35	0	33.6
1	1	85	66	29	0	26.6
2	8	183	64	0	0	23.3
3	1	89	66	23	94	28.1
4	0	137	40	35	168	43.1

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Original Data:

	RI	Na	Mg	Al	Si	K	Ca	Ba	Fe	Type_of_glass
0	1.52101	13.64	4.49	1.10	71.78	0.06	8.75	0.0	0.0	1
1	1.51761	13.89	3.60	1.36	72.73	0.48	7.83	0.0	0.0	1
2	1.51618	13.53	3.55	1.54	72.99	0.39	7.78	0.0	0.0	1
3	1.51766	13.21	3.69	1.29	72.61	0.57	8.22	0.0	0.0	1
4	1.51742	13.27	3.62	1.24	73.08	0.55	8.07	0.0	0.0	1

Glass class distribution:

Type_of_glass

1	70
2	76
3	17
5	13
6	9
7	29

Name: count, dtype: int64

Diabetes class distribution:

Outcome

0	500
1	268

Name: count, dtype: int64

Glass missing values:

RI	0
Na	0
Mg	0
Al	0
Si	0
K	0
Ca	0
Ba	0
Fe	0

dtype: int64

Diabetes missing values:

Pregnancies	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0

dtype: int64

Glass normalized sample:

RI	Na	Mg	Al	Si	K	Ca	Ba	\
----	----	----	----	----	---	----	----	---

0	0.520193	0.387970	0.797327	0.151899	0.389286	0.044444	0.421154	0.0
1	0.351185	0.300752	0.412027	0.481013	0.514286	0.222222	0.413462	0.0
2	0.217296	0.390977	0.786192	0.316456	0.510714	0.207407	0.244231	0.0
3	0.464004	0.413534	0.832962	0.177215	0.392857	0.066667	0.367308	0.0
4	0.351624	0.514286	0.000000	0.553797	0.523214	0.000000	0.489423	0.0

	Fe
0	0.058824
1	0.000000
2	0.000000
3	0.137255
4	0.000000

Diabetes normalized sample:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI \
0	0.411765	0.900709	0.612245	0.490566	0.151099	0.353783
1	0.352941	0.198582	0.551020	0.415094	0.151099	0.265849
2	0.117647	0.347518	0.408163	0.528302	0.141484	0.251534
3	0.117647	0.304965	0.306122	0.396226	0.122253	0.400818
4	0.294118	0.333333	0.510204	0.415094	0.151099	0.216769

	DiabetesPedigreeFunction	Age
0	0.120713	0.333333
1	0.132739	0.350000
2	0.586192	0.216667
3	0.184410	0.050000
4	0.030735	0.450000

```
In [14]: # t-SNE Visualization before RBF Kernel
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
# Split features (X) and labels (y)
X = X_diab_normalized_df.values
y = y_diab.values

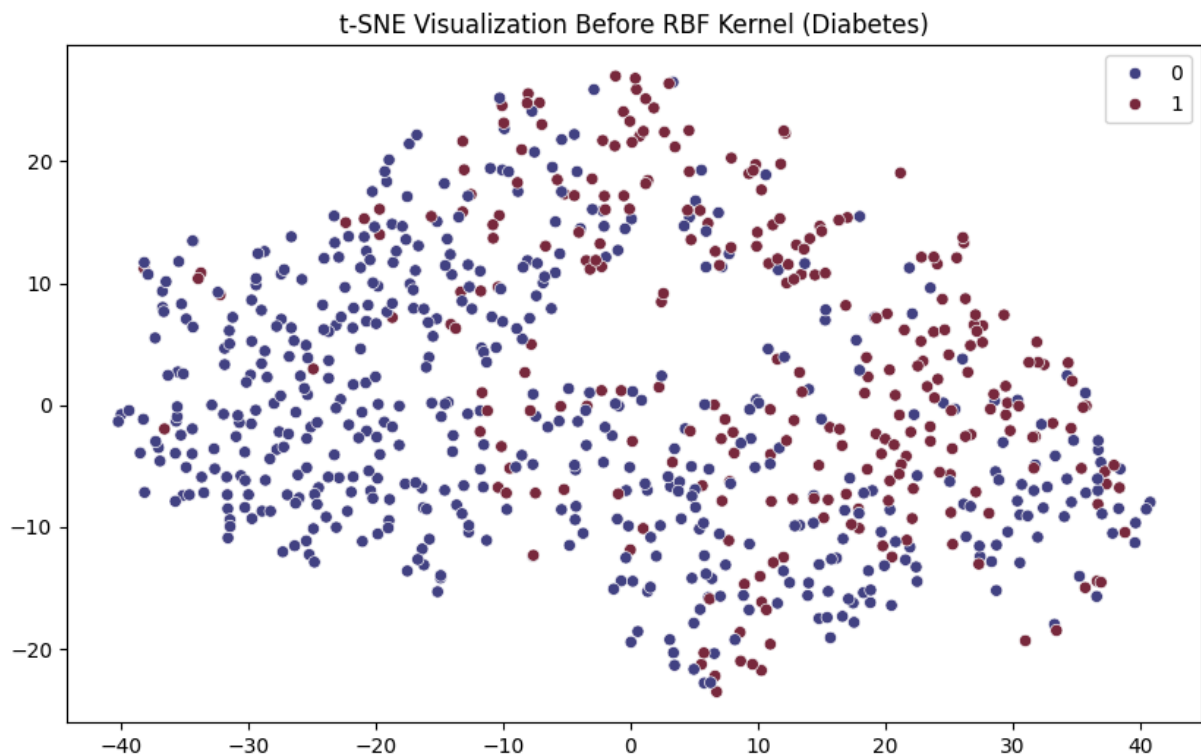
# Print the shapes of X and y
print("X shape:", X.shape)
print("y shape:", y.shape)

# Apply t-SNE to reduce dimensionality to 2 components (2D)
tsne = TSNE(n_components=2, random_state=42, perplexity=30, init="pca", learning_rate=100)
X_tsne = tsne.fit_transform(X)

# Visualize the t-SNE results
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y, palette='icefire', legend="f")
plt.title("t-SNE Visualization Before RBF Kernel (Diabetes)")
plt.show()
```

X shape: (768, 8)

y shape: (768,)



```
In [15]: # t-SNE Visualization before RBF Kernel
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt
import seaborn as sns
# Split features (X) and labels (y)
X = X_glass_normalized_df.values
y = y_glass.iloc[:, 0].values

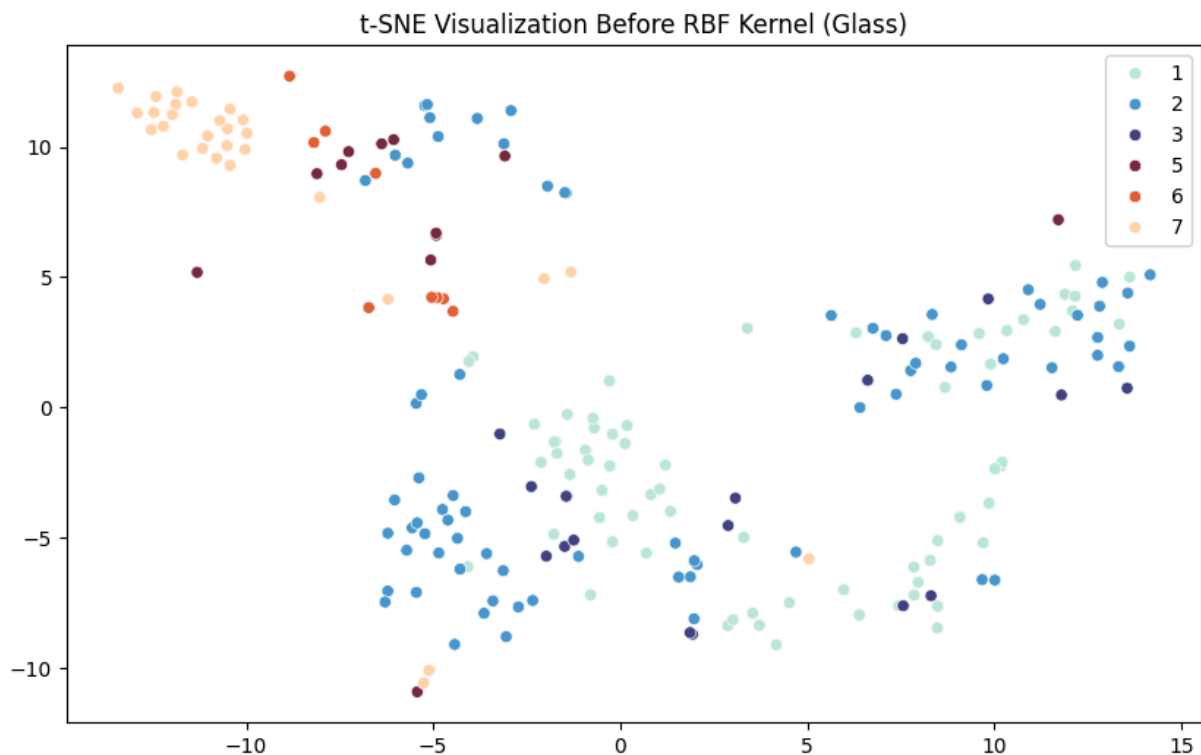
# Print the shapes of X and y
print("X shape:", X.shape)
print("y shape:", y.shape)

# Apply t-SNE to reduce dimensionality to 2 components (2D)
tsne = TSNE(n_components=2, random_state=42, perplexity=30, init="pca", learning_rate=100)
X_tsne = tsne.fit_transform(X)

# Visualize the t-SNE results
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y, palette='icefire', legend="f")
plt.title("t-SNE Visualization Before RBF Kernel (Glass)")
plt.show()
```

X shape: (214, 9)

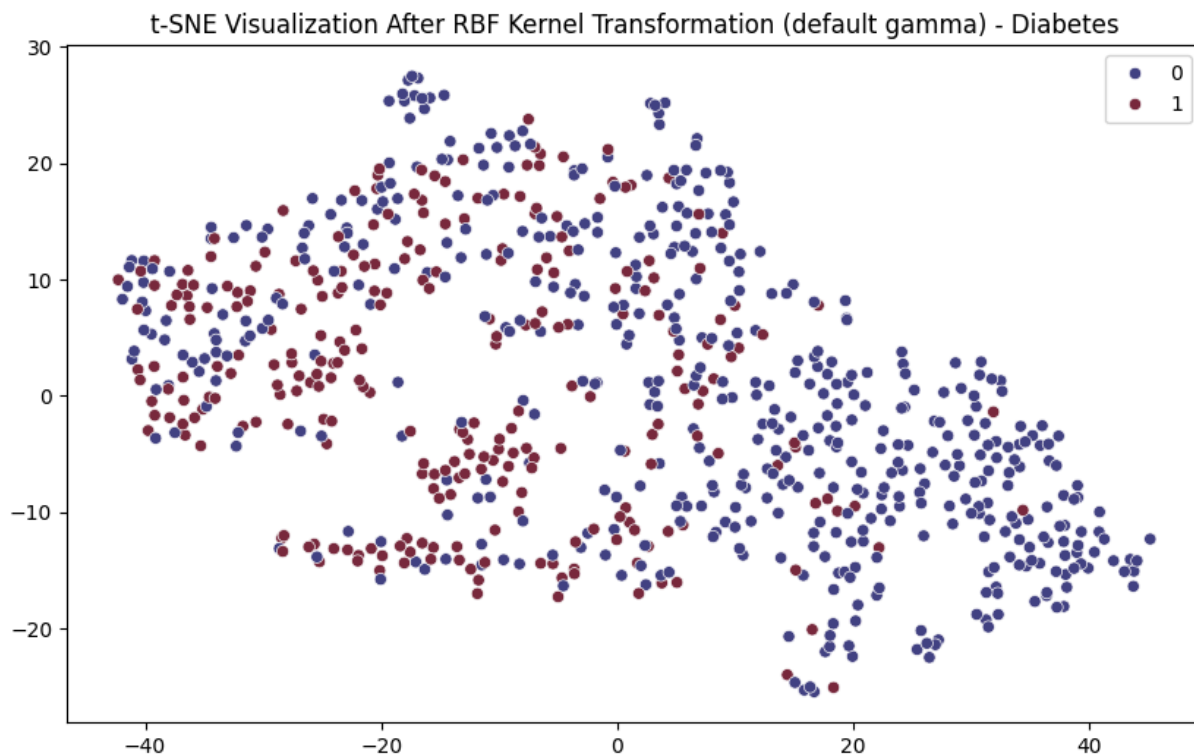
y shape: (214,)



```
In [16]: from sklearn.metrics.pairwise import rbf_kernel
# Apply RBF Kernel using sklearn.metrics.pairwise.rbf_kernel
X_rbf = rbf_kernel(X_diab_normalized_df.values)

# Apply t-SNE to visualize the class distributions after RBF Kernel
X_tsne = TSNE(n_components=2, random_state=42, perplexity=30, init="pca", learning_

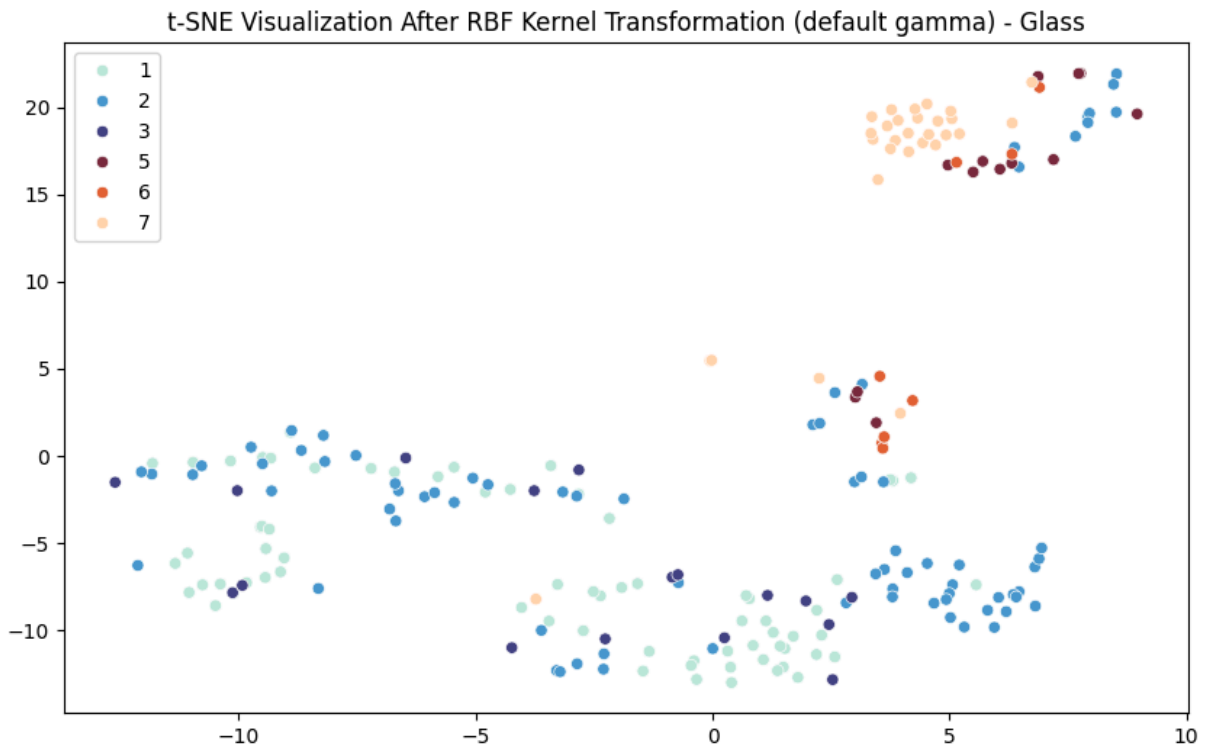
# Visualize the t-SNE results
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y_diab.values, palette="icefire")
plt.title("t-SNE Visualization After RBF Kernel Transformation (default gamma) - Di
plt.show()
```

```
In [17]: from sklearn.metrics.pairwise import rbf_kernel
# Apply RBF Kernel using sklearn.metrics.pairwise.rbf_kernel
X_rbf = rbf_kernel(X_glass_normalized_df.values)

# Apply t-SNE to visualize the class distributions after RBF Kernel
X_tsne = TSNE(n_components=2, random_state=42, perplexity=30, init="pca", learning_

# Visualize the t-SNE results
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y_glass.iloc[:, 0].values, palette=
plt.title("t-SNE Visualization After RBF Kernel Transformation (default gamma) - Gl
plt.show()
```



```
In [18]: import numpy as np
from sklearn.model_selection import train_test_split
print("Linear Perceptron Before RBF Kernel (Diabetes)")
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score, GridSearchCV
from sklearn.metrics import make_scorer, f1_score
from sklearn.linear_model import SGDClassifier
# set up the train and test splits

X_test = Xd_test_norm
y_test = yd_test

print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_test: {y_test.shape}")
# Define the Perceptron model
model = SGDClassifier(loss="perceptron", random_state=42)

# Define the parameter grid for tuning the max_iter parameter
param_grid = {"max_iter": [100, 300, 500, 1000, 2000, 5000]}

# Define the Repeated K-Fold Cross-Validation with 10 folds and 3 repeats
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)

# Use GridSearchCV to find the best parameters using cross-validation
scorer = make_scorer(f1_score, average="weighted")
grid = GridSearchCV(model, param_grid=param_grid, scoring=scorer, cv=cv, n_jobs=-1)
grid.fit(Xd_train_norm, yd_train)

# Get the best parameter value
best_max_iter_diab_before = grid.best_params_["max_iter"]

print(f"Best max_iter: {best_max_iter_diab_before}")
# Get the best estimator (model) found by GridSearchCV
```

```

best_model_diab_before = grid.best_estimator_

# Cross-validation scores for accuracy
scores = cross_val_score(best_model_diab_before, Xd_train_norm, yd_train, scoring=s

# Calculate the weighted F1-score across all classes
weighted_f1_cv = np.mean(scores)
y_pred_test = best_model_diab_before.predict(X_test)
weighted_f1_test = f1_score(y_test, y_pred_test, average="weighted")

# Output the results
print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

```

Linear Perceptron Before RBF Kernel (Diabetes)

Shape of X_test: (231, 8)

Shape of y_test: (231,)

Best max_iter: 100

Cross Validation Scores: [0.72325405 0.69216259 0.55883058 0.53859053 0.80867856 0.71887097

0.71751333 0.72635231 0.76076156 0.61482493 0.65831801 0.69357989

0.65048348 0.74348324 0.68793867]

Average CV Score: 0.6862428458450877

Test Weighted F1: 0.5962967882716113

```

In [19]: import numpy as np
from sklearn.model_selection import train_test_split
print("Linear Perceptron Before RBF Kernel (Glass)")
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score, GridS
from sklearn.metrics import make_scorer, f1_score
from sklearn.linear_model import SGDClassifier
# set up the train and test splits

X_test = Xg_test_norm
y_test = yg_test

print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_test: {y_test.shape}")
# Define the Perceptron model
model = SGDClassifier(loss="perceptron", random_state=42)

# Define the parameter grid for tuning the max_iter parameter
param_grid = {"max_iter": [100, 300, 500, 1000, 2000, 5000]}

# Define the Repeated K-Fold Cross-Validation with 10 folds and 3 repeats
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)

# Use GridSearchCV to find the best parameters using cross-validation
scorer = make_scorer(f1_score, average="weighted")
grid = GridSearchCV(model, param_grid=param_grid, scoring=scorer, cv=cv, n_jobs=-1)
grid.fit(Xg_train_norm, yg_train)

# Get the best parameter value
best_max_iter_glass_before = grid.best_params_["max_iter"]

```

```

print(f"Best max_iter: {best_max_iter_glass_before}")
# Get the best estimator (model) found by GridSearchCV
best_model_glass_before = grid.best_estimator_

# Cross-validation scores for accuracy
scores = cross_val_score(best_model_glass_before, Xg_train_norm, yg_train, scoring=

# Calculate the weighted F1-score across all classes
weighted_f1_cv = np.mean(scores)
y_pred_test = best_model_glass_before.predict(X_test)
weighted_f1_test = f1_score(y_test, y_pred_test, average="weighted")

# Output the results
print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

```

Linear Perceptron Before RBF Kernel (Glass)

Shape of X_test: (65, 9)

Shape of y_test: (65,)

Best max_iter: 100

Cross Validation Scores: [0.55809524 0.3984127 0.44330484 0.42798535 0.35803851 0.33997114

0.4702381 0.44022989 0.21282051 0.3862069 0.36571429 0.45616162

0.3844189 0.41196581 0.49042146]

Average CV Score: 0.40959901613924593

Test Weighted F1: 0.5835897435897437

```

In [20]: from sklearn.metrics.pairwise import rbf_kernel
from sklearn.model_selection import RepeatedStratifiedKFold, GridSearchCV, cross_val_score
from sklearn.metrics import make_scorer, f1_score
from sklearn.linear_model import SGDClassifier
import numpy as np
print("Linear Perceptron After RBF Kernel (Diabetes)")
# Apply the RBF kernel transformation
gamma_default = 1.0 / Xd_train_norm.shape[1]
X_train_rbf = rbf_kernel(Xd_train_norm.values, Xd_train_norm.values, gamma=gamma_default)

# Define the Linear Perceptron model
rbf_li_perceptron_2 = SGDClassifier(loss="perceptron", random_state=42)
# Define the parameter grid for tuning the max_iter parameter
param_grid = {"max_iter": [100, 300, 500, 1000, 2000, 5000]}

# Define the Repeated K-Fold Cross-Validation with 10 folds and 3 repeats
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)

# Use GridSearchCV to find the best parameters using cross-validation
scorer = make_scorer(f1_score, average="weighted")
grid = GridSearchCV(rbf_li_perceptron_2, param_grid=param_grid, scoring=scorer, cv=cv)

# Fit the model on the RBF transformed data
grid.fit(X_train_rbf, yd_train)

# Get the best parameters
best_max_iter_diab_rbf = grid.best_params_["max_iter"]

```

```

print(f"Best max_iter: {best_max_iter_diab_rbf}")
# Get the best estimator (model) found by GridSearchCV
best_model_diab_rbf = grid.best_estimator_

# Calculate the weighted F1-score across all classes
scores = cross_val_score(best_model_diab_rbf, X_train_rbf, yd_train, scoring=scorer)
weighted_f1_cv = np.mean(scores)

X_test_rbf = rbf_kernel(Xd_test_norm.values, Xd_train_norm.values, gamma=gamma_defa
y_pred_test = best_model_diab_rbf.predict(X_test_rbf)
weighted_f1_test = f1_score(yd_test, y_pred_test, average="weighted")

print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

```

Linear Perceptron After RBF Kernel (Diabetes)

Best max_iter: 300

Cross Validation Scores: [0.69138011 0.50533584 0.53859053 0.74382938 0.51745076 0.50533584

0.76918898 0.63210769 0.67164487 0.6497293 0.64995871 0.77282177
0.68903123 0.66121351 0.7638698]

Average CV Score: 0.6507658889903812

Test Weighted F1: 0.5308966393621637

```

In [21]: from sklearn.metrics.pairwise import rbf_kernel
from sklearn.model_selection import RepeatedStratifiedKFold, GridSearchCV, cross_val_score
from sklearn.metrics import make_scorer, f1_score
from sklearn.linear_model import SGDClassifier
import numpy as np
print("Linear Perceptron After RBF Kernel (Glass)")
# Apply the RBF kernel transformation
gamma_default = 1.0 / Xg_train_norm.shape[1]
X_train_rbf = rbf_kernel(Xg_train_norm.values, gamma=gamma_default)

# Define the Linear Perceptron model
rbf_li_perceptron_2 = SGDClassifier(loss="perceptron", random_state=42)
# Define the parameter grid for tuning the max_iter parameter
param_grid = {"max_iter": [100, 300, 500, 1000, 2000, 5000]}

# Define the Repeated K-Fold Cross-Validation with 10 folds and 3 repeats
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)

# Use GridSearchCV to find the best parameters using cross-validation
scorer = make_scorer(f1_score, average="weighted")
grid = GridSearchCV(rbf_li_perceptron_2, param_grid=param_grid, scoring=scorer, cv=cv)

# Fit the model on the RBF transformed data
grid.fit(X_train_rbf, yg_train)

# Get the best parameters
best_max_iter_glass_rbf = grid.best_params_["max_iter"]

print(f"Best max_iter: {best_max_iter_glass_rbf}")
# Get the best estimator (model) found by GridSearchCV
best_model_glass_rbf = grid.best_estimator_

```

```

# Calculate the weighted F1-score across all classes
scores = cross_val_score(best_model_glass_rbf, X_train_rbf, yg_train, scoring='score_weighted_f1_cv' = np.mean(scores)

X_test_rbf = rbf_kernel(Xg_test_norm.values, Xg_train_norm.values, gamma=gamma_defa
y_pred_test = best_model_glass_rbf.predict(X_test_rbf)
weighted_f1_test = f1_score(yg_test, y_pred_test, average="weighted")

print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

```

Linear Perceptron After RBF Kernel (Glass)

Best max_iter: 100

Cross Validation Scores: [0.15213675 0.22222222 0.2989899 0.10705128 0.18606322 0.02777778

0.16666667 0.275 0.03137255 0.14700544 0.08122605 0.19674797
0.33055556 0.3094686 0.14700544]

Average CV Score: 0.17861929548374011

Test Weighted F1: 0.18496503496503494

```

In [22]: from sklearn.metrics.pairwise import rbf_kernel
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import make_scorer, f1_score
import numpy as np

# Apply RBF Kernel using sklearn.metrics.pairwise.rbf_kernel
X_rbf = rbf_kernel(X_diab_normalized_df.values, gamma=1.0 / Xd_train_norm.shape[1])

# Print the shapes of X_rbf
print("Initial X_rbf shape:", X_rbf.shape)

# Apply t-SNE to visualize the class distributions after RBF Kernel
X_tsne = TSNE(n_components=2, random_state=42, perplexity=30, init="pca", learning_

# Visualize the t-SNE results
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y_diab.values, palette="icefire")
plt.title("t-SNE Visualization After RBF Kernel Transformation (default gamma) - Di
plt.show()

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
scorer = make_scorer(f1_score, average="weighted")

gammas = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
best_gamma_diab = None
best_score = -1

for g in gammas:
    X_train_rbf = rbf_kernel(Xd_train_norm.values, gamma=g)
    m = SGDClassifier(loss="perceptron", random_state=42, max_iter=best_max_iter_di
    s = cross_val_score(m, X_train_rbf, yd_train, scoring=scorer, cv=cv, n_jobs=-1)
    if np.mean(s) > best_score:
        best_score = np.mean(s)

```

```

best_gamma_diab = g

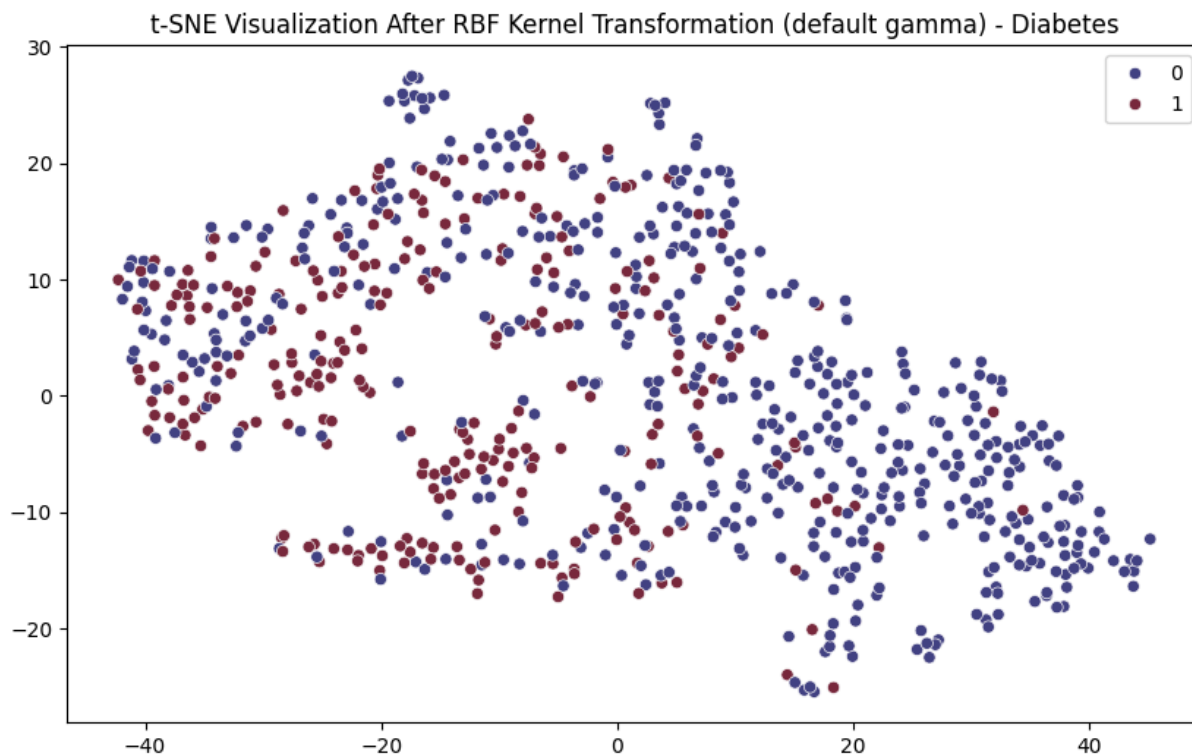
X_rbf = rbf_kernel(X_diab_normalized_df.values, gamma=best_gamma_diab)
print("Tuned X_rbf shape:", X_rbf.shape)

X_tsne = TSNE(n_components=2, random_state=42, perplexity=30, init="pca", learning_

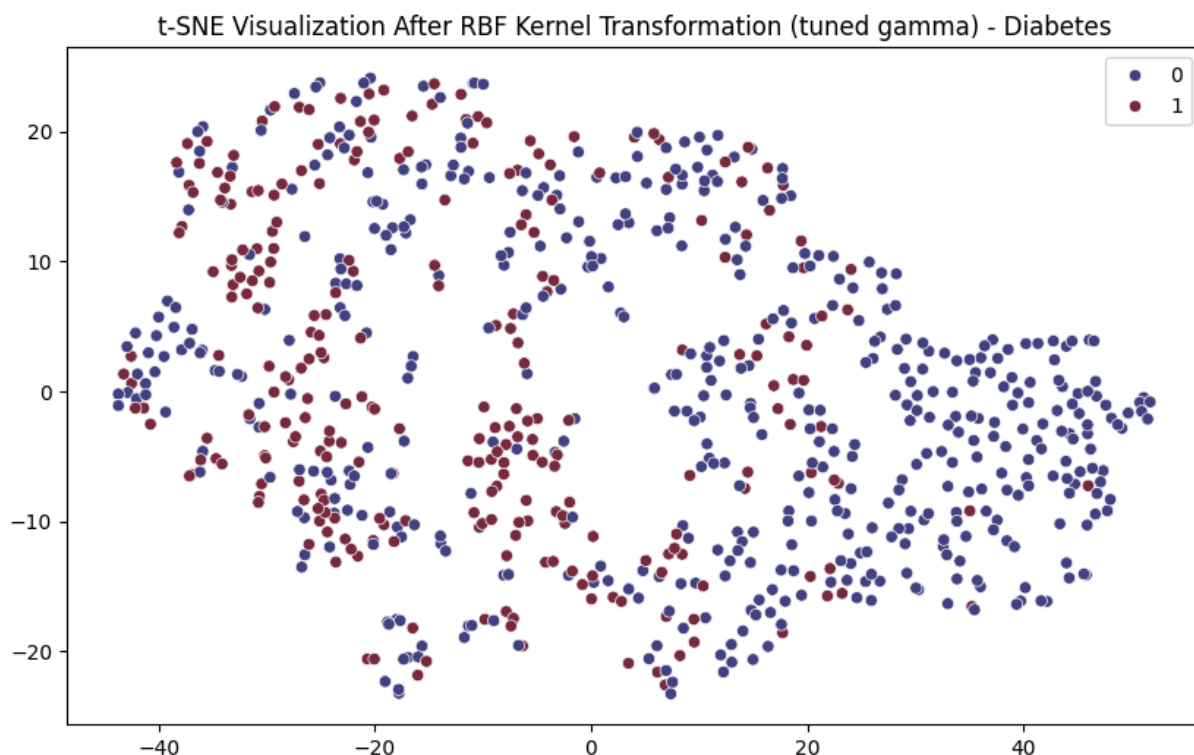
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y_diab.values, palette="icefire")
plt.title("t-SNE Visualization After RBF Kernel Transformation (tuned gamma) - Diab")
plt.show()

```

Initial X_rbf shape: (768, 768)



Tuned X_rbf shape: (768, 768)



```
In [23]: from sklearn.metrics.pairwise import rbf_kernel
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import make_scorer, f1_score
import numpy as np

# Apply RBF Kernel using sklearn.metrics.pairwise.rbf_kernel
X_rbf = rbf_kernel(X_glass_normalized_df.values, gamma=1.0 / Xg_train_norm.shape[1])

# Print the shapes of X_rbf
print("Initial X_rbf shape:", X_rbf.shape)

# Apply t-SNE to visualize the class distributions after RBF Kernel
X_tsne = TSNE(n_components=2, random_state=42, perplexity=30, init="pca", learning_

# Visualize the t-SNE results
plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y_glass.iloc[:, 0].values, palette="magma")
plt.title("t-SNE Visualization After RBF Kernel Transformation (default gamma) - G1")
plt.show()

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
scorer = make_scorer(f1_score, average="weighted")

gammas = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
best_gamma_glass = None
best_score = -1

for g in gammas:
    X_train_rbf = rbf_kernel(Xg_train_norm.values, gamma=g)
    m = SGDClassifier(loss="perceptron", random_state=42, max_iter=best_max_iter_g1)
    s = cross_val_score(m, X_train_rbf, yg_train, scoring=scorer, cv=cv, n_jobs=-1)
```



```

    if np.mean(s) > best_score:
        best_score = np.mean(s)
        best_gamma_glass = g

X_rbf = rbf_kernel(X_glass_normalized_df.values, gamma=best_gamma_glass)
print("Tuned X_rbf shape:", X_rbf.shape)

X_tsne = TSNE(n_components=2, random_state=42, perplexity=30, init="pca", learning_

plt.figure(figsize=(10, 6))
sns.scatterplot(x=X_tsne[:, 0], y=X_tsne[:, 1], hue=y_glass.iloc[:, 0].values, palette=
plt.title("t-SNE Visualization After RBF Kernel Transformation (tuned gamma) - Glas
plt.show()

print("Logistic Regression Before RBF Kernel (Diabetes)")
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
scorer = make_scorer(f1_score, average="weighted")

model = SGDClassifier(loss="log_loss", random_state=42, class_weight=None)
param_grid = {"max_iter": [100, 300, 500, 1000, 2000, 5000]}

grid = GridSearchCV(model, param_grid=param_grid, scoring=scorer, cv=cv, n_jobs=-1)
grid.fit(Xd_train_norm, yd_train)

best_max_iter_log_diab_before = grid.best_params_["max_iter"]
print("Best max_iter:", best_max_iter_log_diab_before)

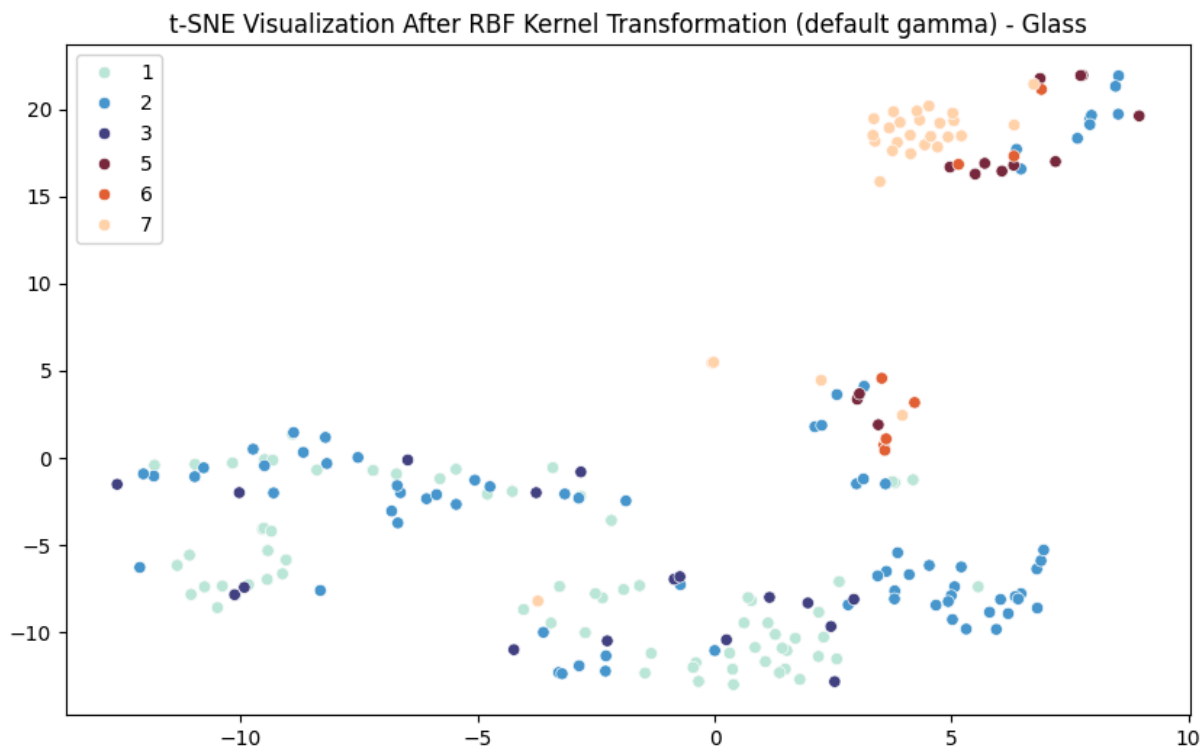
best_log_diab_before = grid.best_estimator_
scores = cross_val_score(best_log_diab_before, Xd_train_norm, yd_train, scoring=sco
weighted_f1_cv = np.mean(scores)

y_pred_test = best_log_diab_before.predict(Xd_test_norm)
weighted_f1_test = f1_score(yd_test, y_pred_test, average="weighted")

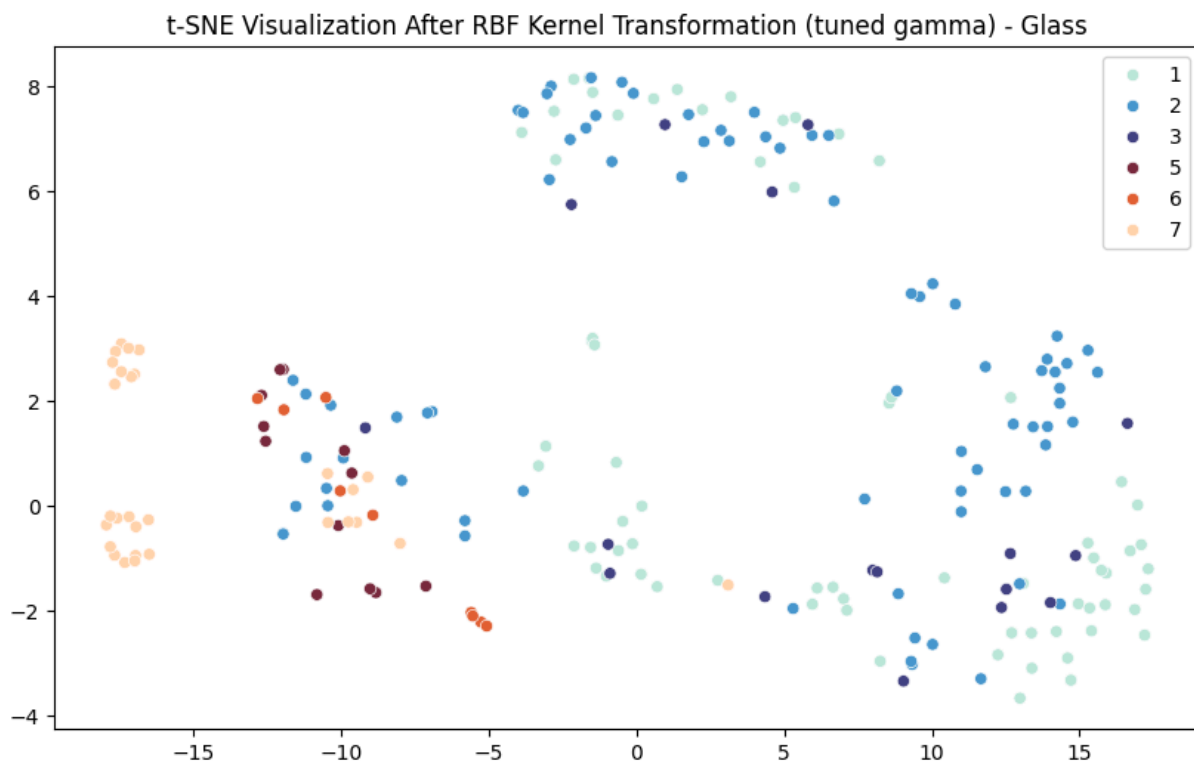
print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

```

Initial X_rbf shape: (214, 214)



Tuned X_rbf shape: (214, 214)



Logistic Regression Before RBF Kernel (Diabetes)

Best max_iter: 100

Cross Validation Scores: [0.72616592 0.66435598 0.78433669 0.79664087 0.82612689 0.74075929

0.74218589 0.66489459 0.80680766 0.7443083 0.81757135 0.66931217

0.75138465 0.59688702 0.72410231]

Average CV Score: 0.7370559718548978

Test Weighted F1: 0.6926117360899969

```

In [24]: print("Logistic Regression After RBF Kernel (Diabetes)")
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
scorer = make_scorer(f1_score, average="weighted")

gamma_default = 1.0 / Xd_train_norm.shape[1]
X_train_rbf = rbf_kernel(Xd_train_norm.values, Xd_train_norm.values, gamma=gamma_de

model = SGDClassifier(loss="log_loss", random_state=42, class_weight=None)
param_grid = {"max_iter": [100, 300, 500, 1000, 2000, 5000]}

grid = GridSearchCV(model, param_grid=param_grid, scoring=scorer, cv=cv, n_jobs=-1)
grid.fit(X_train_rbf, yd_train)

best_max_iter_log_diab_rbf = grid.best_params_["max_iter"]
print("Best max_iter:", best_max_iter_log_diab_rbf)

best_log_diab_rbf = grid.best_estimator_
scores = cross_val_score(best_log_diab_rbf, X_train_rbf, yd_train, scoring=scorer,
weighted_f1_cv = np.mean(scores)

X_test_rbf = rbf_kernel(Xd_test_norm.values, Xd_train_norm.values, gamma=gamma_defa
y_pred_test = best_log_diab_rbf.predict(X_test_rbf)
weighted_f1_test = f1_score(yd_test, y_pred_test, average="weighted")

print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

print("Tuning gamma for Logistic Regression (Diabetes)")
gammas = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
best_gamma_log_diab = None
best_score = -1

for g in gammas:
    X_train_rbf_g = rbf_kernel(Xd_train_norm.values, Xd_train_norm.values, gamma=g)
    m = SGDClassifier(loss="log_loss", random_state=42, class_weight=None, max_iter
    s = cross_val_score(m, X_train_rbf_g, yd_train, scoring=scorer, cv=cv, n_jobs=-
    if np.mean(s) > best_score:
        best_score = np.mean(s)
        best_gamma_log_diab = g

print("Best gamma:", best_gamma_log_diab)
print("Best gamma CV F1:", best_score)

```

Logistic Regression After RBF Kernel (Diabetes)

Best max_iter: 100

Cross Validation Scores: [0.67491463 0.55862248 0.68048219 0.53859053 0.21654747 0.5379938

0.59999217 0.79137368 0.68986595 0.51745076 0.81757135 0.76466158
0.6786434 0.62721318 0.71564326]

Average CV Score: 0.6273044283230583

Test Weighted F1: 0.7348651348651348

Tuning gamma for Logistic Regression (Diabetes)

Best gamma: 10.0

Best gamma CV F1: 0.6902675091288643

```
In [25]: print("Logistic Regression Before RBF Kernel (Glass)")
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
scorer = make_scorer(f1_score, average="weighted")

model = SGDClassifier(loss="log_loss", random_state=42, class_weight=None)
param_grid = {"max_iter": [100, 300, 500, 1000, 2000, 5000]}

grid = GridSearchCV(model, param_grid=param_grid, scoring=scorer, cv=cv, n_jobs=-1)
grid.fit(Xg_train_norm, yg_train)

best_max_iter_log_glass_before = grid.best_params_["max_iter"]
print("Best max_iter:", best_max_iter_log_glass_before)

best_log_glass_before = grid.best_estimator_
scores = cross_val_score(best_log_glass_before, Xg_train_norm, yg_train, scoring=scorer, cv=cv)
weighted_f1_cv = np.mean(scores)

y_pred_test = best_log_glass_before.predict(Xg_test_norm)
weighted_f1_test = f1_score(yg_test, y_pred_test, average="weighted")

print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)
```

Logistic Regression Before RBF Kernel (Glass)

Best max_iter: 100

Cross Validation Scores: [0.28055556 0.3010101 0.47962963 0.37171717 0.47872817 0.52234289
0.43365079 0.31813865 0.3988604 0.50935961 0.50888889 0.41616162
0.36753813 0.52827381 0.4702623]

Average CV Score: 0.4256745144096038

Test Weighted F1: 0.47325195130073183

```
In [26]: print("Logistic Regression After RBF Kernel (Glass)")
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
scorer = make_scorer(f1_score, average="weighted")

gamma_default = 1.0 / Xg_train_norm.shape[1]
X_train_rbf = rbf_kernel(Xg_train_norm.values, Xg_train_norm.values, gamma=gamma_default)

model = SGDClassifier(loss="log_loss", random_state=42, class_weight=None)
param_grid = {"max_iter": [100, 300, 500, 1000, 2000, 5000]}

grid = GridSearchCV(model, param_grid=param_grid, scoring=scorer, cv=cv, n_jobs=-1)
grid.fit(X_train_rbf, yg_train)

best_max_iter_log_glass_rbf = grid.best_params_["max_iter"]
print("Best max_iter:", best_max_iter_log_glass_rbf)

best_log_glass_rbf = grid.best_estimator_
scores = cross_val_score(best_log_glass_rbf, X_train_rbf, yg_train, scoring=scorer, cv=cv)
weighted_f1_cv = np.mean(scores)

X_test_rbf = rbf_kernel(Xg_test_norm.values, Xg_train_norm.values, gamma=gamma_default)
y_pred_test = best_log_glass_rbf.predict(X_test_rbf)
weighted_f1_test = f1_score(yg_test, y_pred_test, average="weighted")
```

```

print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

print("Tuning gamma for Logistic Regression (Glass)")
gammas = [0.001, 0.01, 0.1, 1.0, 10.0, 100.0]
best_gamma_log_glass = None
best_score = -1

for g in gammas:
    X_train_rbf_g = rbf_kernel(Xg_train_norm.values, Xg_train_norm.values, gamma=g)
    m = SGDClassifier(loss="log_loss", random_state=42, class_weight=None, max_iter=
    s = cross_val_score(m, X_train_rbf_g, yg_train, scoring=scorer, cv=cv, n_jobs=-
    if np.mean(s) > best_score:
        best_score = np.mean(s)
        best_gamma_log_glass = g

print("Best gamma:", best_gamma_log_glass)
print("Best gamma CV F1:", best_score)

```

Logistic Regression After RBF Kernel (Glass)

Best max_iter: 100

Cross Validation Scores: [0.2989899 0.22273425 0.20718954 0.08444444 0.01939655 0.36590909

0.16666667 0.28106061 0.24973545 0.21367449 0.26432749 0.03232323

0.15818182 0.16666667 0.15908046]

Average CV Score: 0.19269204385765215

Test Weighted F1: 0.3337782805429864

Tuning gamma for Logistic Regression (Glass)

Best gamma: 10.0

Best gamma CV F1: 0.5880868832628189

```

In [28]: print("MLPClassifier (Diabetes)")
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV, cross_val_score, RepeatedStratifi
from sklearn.metrics import f1_score, make_scorer

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
scorer = make_scorer(f1_score, average="weighted")

mlp = MLPClassifier(random_state=42)
param_grid = {
    "max_iter": [200, 500, 1000, 2000],
    "n_iter_no_change": [5, 10, 20]
}

grid = GridSearchCV(mlp, param_grid=param_grid, scoring=scorer, cv=cv, n_jobs=-1)
grid.fit(Xd_train_norm, yd_train)

print("Best params:", grid.best_params_)
best_mlp_diab = grid.best_estimator_

scores = cross_val_score(best_mlp_diab, Xd_train_norm, yd_train, scoring=scorer, cv
weighted_f1_cv = np.mean(scores)

```

```

y_pred_test = best_mlp_diab.predict(Xd_test_norm)
weighted_f1_test = f1_score(yd_test, y_pred_test, average="weighted")

print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

```

MLPClassifier (Diabetes)

Best params: {'max_iter': 1000, 'n_iter_no_change': 10}

Cross Validation Scores: [0.7183642 0.77071189 0.73355073 0.7901523 0.81180437 0.74075929

0.79175183 0.78748842 0.80165063 0.76191811 0.82803392 0.76619018
0.74884995 0.80680766 0.75971278]

Average CV Score: 0.7745164174634538

Test Weighted F1: 0.7313771134525852

```

In [29]: print("MLPClassifier (Glass)")

from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import GridSearchCV, cross_val_score, RepeatedStratifiedKFold
from sklearn.metrics import f1_score, make_scorer

cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)
scorer = make_scorer(f1_score, average="weighted")

mlp = MLPClassifier(random_state=42)
param_grid = {
    "max_iter": [200, 500, 1000, 2000],
    "n_iter_no_change": [5, 10, 20]
}

grid = GridSearchCV(mlp, param_grid=param_grid, scoring=scorer, cv=cv, n_jobs=-1)
grid.fit(Xg_train_norm, yg_train)

print("Best params:", grid.best_params_)
best_mlp_glass = grid.best_estimator_

scores = cross_val_score(best_mlp_glass, Xg_train_norm, yg_train, scoring=scorer, cv=cv)
weighted_f1_cv = np.mean(scores)

y_pred_test = best_mlp_glass.predict(Xg_test_norm)
weighted_f1_test = f1_score(yg_test, y_pred_test, average="weighted")

print("Cross Validation Scores: ", scores)
print("Average CV Score: ", weighted_f1_cv)
print("Test Weighted F1: ", weighted_f1_test)

```

MLPClassifier (Glass)

```

/usr/local/lib/python3.12/dist-packages/sklearn/neural_network/_multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (2000) reached and the optimization hasn't converged yet.
  warnings.warn(

```

Best params: {'max_iter': 2000, 'n_iter_no_change': 5}
 Cross Validation Scores: [0.5955191 0.7510582 0.64363929 0.73428571 0.71931553 0.71428571
 0.73492063 0.57856913 0.68066667 0.66995074 0.53781513 0.74061404
 0.61396609 0.73226545 0.66724157]
 Average CV Score: 0.6742741989413147
 Test Weighted F1: 0.723600982817108

```
In [30]: print("Logistic Regression Feature Significance (Diabetes)")

import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import f1_score

model = SGDClassifier(loss="log_loss", random_state=42, class_weight=None, max_iter=1000)
model.fit(Xd_train_norm, yd_train)

coefs = model.coef_
importance = np.mean(np.abs(coefs), axis=0)

feature_names = Xd_train_norm.columns.tolist()
ranked = sorted(zip(feature_names, importance), key=lambda x: x[1], reverse=True)

top5 = ranked[:5]
bottom5 = ranked[-5:]

print("Top 5 predictors:", top5)
print("Bottom 5 predictors:", bottom5)

drop_features = [name for name, _ in bottom5]
Xd_train_drop = Xd_train_norm.drop(columns=drop_features)
Xd_test_drop = Xd_test_norm.drop(columns=drop_features)

model2 = SGDClassifier(loss="log_loss", random_state=42, class_weight=None, max_iter=1000)
model2.fit(Xd_train_drop, yd_train)

pred = model2.predict(Xd_test_drop)
f1 = f1_score(yd_test, pred, average="weighted")
print("Weighted F1 after dropping 5 least significant:", f1)
```

Logistic Regression Feature Significance (Diabetes)
 Top 5 predictors: [('Glucose', np.float64(6.374896052955271)), ('BMI', np.float64(5.877151421942416)), ('Pregnancies', np.float64(2.6999309213972587)), ('DiabetesPedigreeFunction', np.float64(2.1580983808314977)), ('SkinThickness', np.float64(1.1396201300137694))]
 Bottom 5 predictors: [('DiabetesPedigreeFunction', np.float64(2.1580983808314977)), ('SkinThickness', np.float64(1.1396201300137694)), ('BloodPressure', np.float64(0.47230252163273007)), ('Age', np.float64(0.4396166331547441)), ('Insulin', np.float64(0.008688045888245536))]
 Weighted F1 after dropping 5 least significant: 0.7170516805903444

```
In [31]: print("Logistic Regression Feature Significance (Glass)")

import numpy as np
from sklearn.linear_model import SGDClassifier
from sklearn.metrics import f1_score
```

```

model = SGDClassifier(loss="log_loss", random_state=42, class_weight=None, max_iter=1000)
model.fit(Xg_train_norm, yg_train)

coefs = model.coef_
importance = np.mean(np.abs(coefs), axis=0)

feature_names = Xg_train_norm.columns.tolist()
ranked = sorted(zip(feature_names, importance), key=lambda x: x[1], reverse=True)

top5 = ranked[:5]
bottom5 = ranked[-5:]

print("Top 5 predictors:", top5)
print("Bottom 5 predictors:", bottom5)

drop_features = [name for name, _ in bottom5]
Xg_train_drop = Xg_train_norm.drop(columns=drop_features)
Xg_test_drop = Xg_test_norm.drop(columns=drop_features)

model2 = SGDClassifier(loss="log_loss", random_state=42, class_weight=None, max_iter=1000)
model2.fit(Xg_train_drop, yg_train)

pred = model2.predict(Xg_test_drop)
f1 = f1_score(yg_test, pred, average="weighted")
print("Weighted F1 after dropping 5 least significant:", f1)

```

Logistic Regression Feature Significance (Glass)

Top 5 predictors: [('Al', np.float64(8.837588877683528)), ('Na', np.float64(8.7989531071478)), ('Mg', np.float64(7.231274905136046)), ('Ba', np.float64(6.453255070442004)), ('Si', np.float64(6.033277786864531))]

Bottom 5 predictors: [('Si', np.float64(6.033277786864531)), ('RI', np.float64(5.229896199190987)), ('K', np.float64(4.85555598856241)), ('Ca', np.float64(4.464868338220416)), ('Fe', np.float64(3.540852316681514))]

Weighted F1 after dropping 5 least significant: 0.5461908097973671

```

In [32]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE
from sklearn.neighbors import KNeighborsRegressor # Changed to Regressor

# 3. Train classifier on ORIGINAL features
clf = SGDClassifier(loss="perceptron", random_state=42, max_iter=best_max_iter_diab)
clf.fit(Xd_train_norm.values, yd_train.values)
print(f"Shape of X_true (features): {Xd_train_norm.values.shape}")

# 4. Apply t-SNE for visualization only
tsne = TSNE(n_components=2, random_state=42, perplexity=30, learning_rate=200)
X_tsne = tsne.fit_transform(Xd_train_norm.values)

# 5. Create mesh grid in t-SNE space
x_min, x_max = X_tsne[:, 0].min() - 1, X_tsne[:, 0].max() + 1
y_min, y_max = X_tsne[:, 1].min() - 1, X_tsne[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500),
                     np.linspace(y_min, y_max, 500))

```



```

# 6. Map mesh grid points back to original feature space
# This is tricky because t-SNE is non-invertible.
# Instead, we approximate decision boundaries by predicting nearest neighbor's o

# Fit a KNN regressor to map t-SNE space -> original features
knn_mapper = KNeighborsRegressor(n_neighbors=1) # Changed to Regressor
knn_mapper.fit(X_tsne, Xd_train_norm.values)

# Map each mesh point to nearest original feature vector
mesh_points_original = knn_mapper.predict(np.c_[xx.ravel(), yy.ravel()])

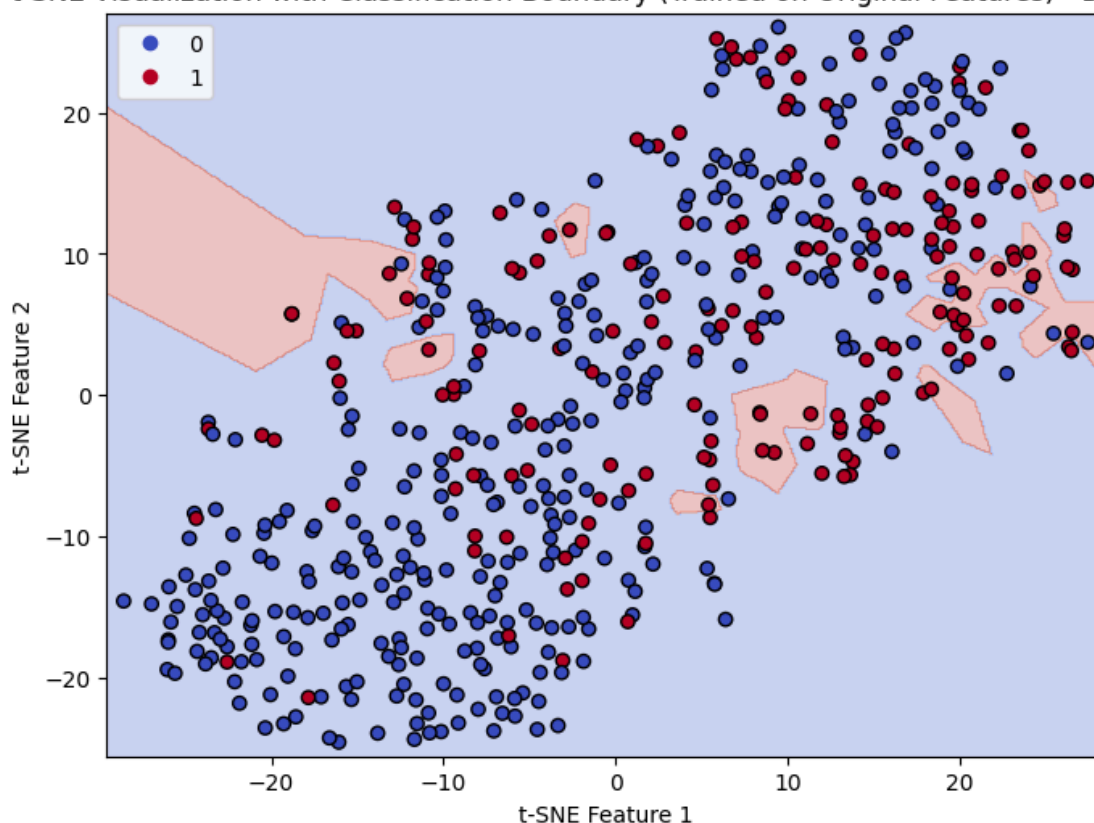
# Predict labels using the classifier trained on original features
Z = clf.predict(mesh_points_original)
Z = Z.reshape(xx.shape)

# 7. Plot decision boundary and t-SNE points
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=yd_train.values, edgecolor='k',
plt.legend(handles=scatter.legend_elements()[0], labels=list(np.unique(yd_train.val
plt.title("t-SNE Visualization with Classification Boundary (Trained on Original Fe
plt.xlabel("t-SNE Feature 1")
plt.ylabel("t-SNE Feature 2")
plt.show()

```

Shape of X_true (features): (537, 8)

t-SNE Visualization with Classification Boundary (Trained on Original Features) - Diabetes



```

In [33]: import numpy as np
import matplotlib.pyplot as plt
from sklearn.manifold import TSNE

```

```

from sklearn.neighbors import KNeighborsRegressor # Changed to Regressor

# 3. Train classifier on ORIGINAL features
clf = SGDClassifier(loss="perceptron", random_state=42, max_iter=best_max_iter_glas
clf.fit(Xg_train_norm.values, yg_train.values)
print(f"Shape of X_true (features): {Xg_train_norm.values.shape}")

# 4. Apply t-SNE for visualization only
tsne = TSNE(n_components=2, random_state=42, perplexity=30, learning_rate=200)
X_tsne = tsne.fit_transform(Xg_train_norm.values)

# 5. Create mesh grid in t-SNE space
x_min, x_max = X_tsne[:, 0].min() - 1, X_tsne[:, 0].max() + 1
y_min, y_max = X_tsne[:, 1].min() - 1, X_tsne[:, 1].max() + 1
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 500),
                     np.linspace(y_min, y_max, 500))

# 6. Map mesh grid points back to original feature space
# This is tricky because t-SNE is non-invertible.
# Instead, we approximate decision boundaries by predicting nearest neighbor's o

# Fit a KNN regressor to map t-SNE space -> original features
knn_mapper = KNeighborsRegressor(n_neighbors=1) # Changed to Regressor
knn_mapper.fit(X_tsne, Xg_train_norm.values)

# Map each mesh point to nearest original feature vector
mesh_points_original = knn_mapper.predict(np.c_[xx.ravel(), yy.ravel()])

# Predict labels using the classifier trained on original features
Z = clf.predict(mesh_points_original)
Z = Z.reshape(xx.shape)

# 7. Plot decision boundary and t-SNE points
plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.3, cmap=plt.cm.coolwarm)
scatter = plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=yg_train.values, edgecolor='k',
plt.legend(handles=scatter.legend_elements()[0], labels=list(np.unique(yg_train.val
plt.title("t-SNE Visualization with Classification Boundary (Trained on Original Fe
plt.xlabel("t-SNE Feature 1")
plt.ylabel("t-SNE Feature 2")
plt.show()

```

Shape of X_true (features): (149, 9)

